

Project 2: Experiment Tracking and Transfer Learning for DNA Accessibility Prediction

Reza Kashkoul Kolouei

Student ID: 800847137

Email: rkashko@siue.edu

Server Account: kashkoul@pc507.emulab.net

April 2026

1 Introduction

In Project 1, I implemented a strict 5-layer 1D Convolutional Neural Network (CNN) for DNA accessibility prediction using DNase-seq data. In Project 2, I extended that original implementation into a more professional experimentation pipeline by adding experiment tracking, TensorBoard logging, automated hyperparameter search, advanced evaluation metrics, model checkpointing, and transfer learning with a frozen backbone.

The objective of this project was not to redesign the model from scratch, but to reuse the existing Project 1 code and dataset and transform it into a reproducible machine learning experiment environment. The final Project 2 implementation preserves the same modular structure and extends it with the following five required components:

1. TensorBoard integration for training and testing visualization
2. Automated hyperparameter grid search
3. Confusion matrix, precision, recall, and F1-score computation
4. Save-best-model checkpointing
5. Frozen-backbone transfer learning on a different organ dataset

2 Computing Environment

All experiments were executed on a remote Linux server through the account `kashkoul@pc507.emulab.net`. The main execution environment was:

- Hostname: `node0.dnaprediction.siu-cs590-490.emulab.net`
- Operating System: Ubuntu 22.04.2 LTS
- CPU: Intel(R) Xeon(R) CPU E5530 @ 2.40GHz
- Number of CPUs: 8
- Memory: 11 GiB RAM
- Root Disk: 63 GB total, 46 GB available during execution

All final experiments were executed from the terminal using the modular project files.

3 Project Foundation from Project 1

The original Project 1 pipeline was reused as the required foundation for Project 2. The reused components were:

- `dataset.py`: dataset building, one-hot encoding, custom PyTorch dataset, balanced Brain dataset construction
- `model.py`: strict 5-layer 1D CNN
- `train.py`: training loop, evaluation loop, experiment handling

The DNA representation remained unchanged from Project 1. Each nucleotide was one-hot encoded as:

$$A = [1, 0, 0, 0], \quad C = [0, 1, 0, 0], \quad G = [0, 0, 1, 0], \quad T = [0, 0, 0, 1]$$

Each sequence of length 174 was transformed into a tensor of shape:

$$174 \times 4$$

and then transposed to:

$$4 \times 174$$

to match the expected input shape for `Conv1D`.

4 Dataset Preparation

4.1 Brain Dataset

The main Brain dataset from Project 1 was reused in Project 2. The five Brain experiments were:

- ENCFF068WQO
- ENCFF109HOA
- ENCFF263JBT
- ENCFF738LML
- ENCFF750GSJ

Following the Project 1 design, the final Brain dataset was balanced by sampling:

- 5,000 positive sequences from each Brain experiment
- 5,000 negative sequences from each Brain experiment

Therefore, the final Brain dataset contained:

$$25,000 \text{ positive} + 25,000 \text{ negative} = 50,000 \text{ total samples}$$

This dataset was randomly split into:

- Training set: 70%
- Validation set: 15%

- Test set: 15%

Although the Brain dataset was split into training, validation, and test subsets, the Project 2 tracking and checkpointing pipeline monitored test loss and test accuracy during training in order to follow the project guideline exactly.

4.2 Heart Dataset for Transfer Learning

For the frozen-backbone experiment, a different organ dataset was introduced:

- Heart experiment: ENCFF586UTZ

The heart sequences were harmonized to length 174 and added into the data loading pipeline through an extended dataset loader. For the heart transfer experiment, the full Heart dataset was split into:

- Training set: 70%
- Test set: 30%

5 Model Architecture

The model architecture remained the same as in Project 1. The network is a strict 5-layer 1D CNN:

1. Conv1D Layer 1 + ReLU + MaxPool
2. Conv1D Layer 2 + ReLU + MaxPool
3. Conv1D Layer 3 + ReLU + MaxPool
4. Dense Layer 1
5. Dense Output Layer with one neuron and Sigmoid activation

Dropout was used in the first fully connected layer to reduce overfitting.

6 Task 1: TensorBoard Integration

The first goal of Project 2 was to integrate `torch.utils.tensorboard` into the training pipeline.

The updated `train.py` implementation logs the following items at every epoch:

- Training Loss
- Test Loss
- Test Accuracy
- Model computational graph using `writer.add_graph()`

This allowed all training dynamics to be tracked visually in TensorBoard instead of relying only on printed terminal output.

In addition to scalar tracking, the final implementation also logged confusion matrix figures and hyperparameter summaries to TensorBoard, which made the experiment pipeline significantly more informative than the Project 1 version.

7 Task 2: Automated Hyperparameter Grid Search

In Project 1, hyperparameter experiments were run manually. In Project 2, this process was automated through a dedicated `grid_search.py` script.

The grid search explored three hyperparameters:

- Learning rate: $\{0.001, 0.0001\}$
- Batch size: $\{16, 64\}$
- Kernel size: $\{5, 11\}$

This produced a total of:

$$2 \times 2 \times 2 = 8 \text{ runs}$$

Each run executed a full training loop automatically and saved:

- TensorBoard logs in a dynamically named folder
- Best model checkpoint
- Accuracy curve
- Loss curve
- Confusion matrix image
- `metrics.txt`

The TensorBoard HParams feature was also used to store the final metrics for comparison.

7.1 Grid Search Results

Table 1: Grid Search Results

Run	LR	Batch	Kernel	Accuracy	Precision	Recall	F1
1	0.001	16	5	0.7291	0.7119	0.7661	0.7380
2	0.001	16	11	0.7677	0.7187	0.7877	0.7516
3	0.001	64	5	0.7483	0.7145	0.8239	0.7653
4	0.001	64	11	0.7372	0.7276	0.7551	0.7411
5	0.0001	16	5	0.7557	0.7487	0.7671	0.7578
6	0.0001	16	11	0.7451	0.7621	0.7099	0.7350
7	0.0001	64	5	0.7500	0.7393	0.7695	0.7541
8	0.0001	64	11	0.7628	0.7690	0.7487	0.7587

7.2 Best Grid Search Configuration

The best overall test accuracy in the grid search was obtained by:

- Learning Rate = 0.001
- Batch Size = 16
- Kernel Size = 11

This configuration achieved:

- Accuracy = 0.7677

- Precision = 0.7187
- Recall = 0.7877
- F1-score = 0.7516

This result is important because it showed that the best configuration in Project 2 was not the same as the best manually selected within-Brain configuration from Project 1. The automated search provided a stronger and more systematic basis for selecting the final pretrained model.

8 Task 3: Advanced Evaluation

Beyond accuracy, the Project 2 pipeline computed more informative classification metrics.

At the end of training, the updated test pipeline computed:

- Confusion Matrix
- Precision
- Recall
- F1-score

The confusion matrix was both saved as an image file and logged directly into TensorBoard using `writer.add_figure()`.

This addition made the evaluation much more informative than simple accuracy alone. In particular, it made it possible to distinguish between high recall behavior and high false-positive behavior.

8.1 Best Grid Search Confusion Matrix

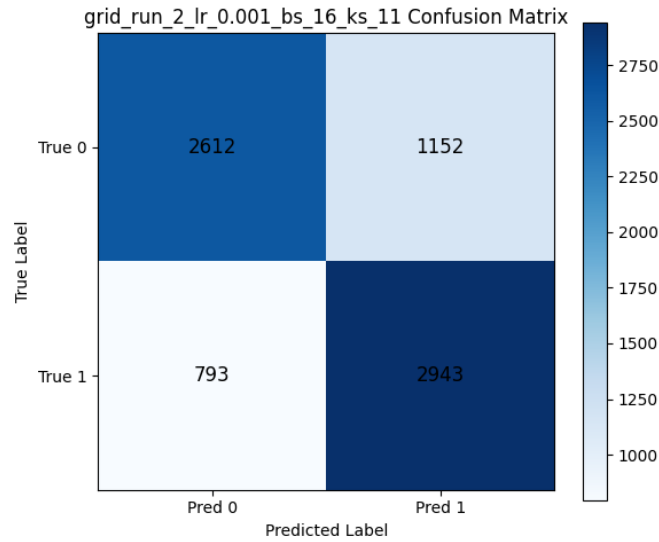


Figure 1: Confusion matrix for the best grid search configuration (Run 2).

8.2 Best Grid Search Curves

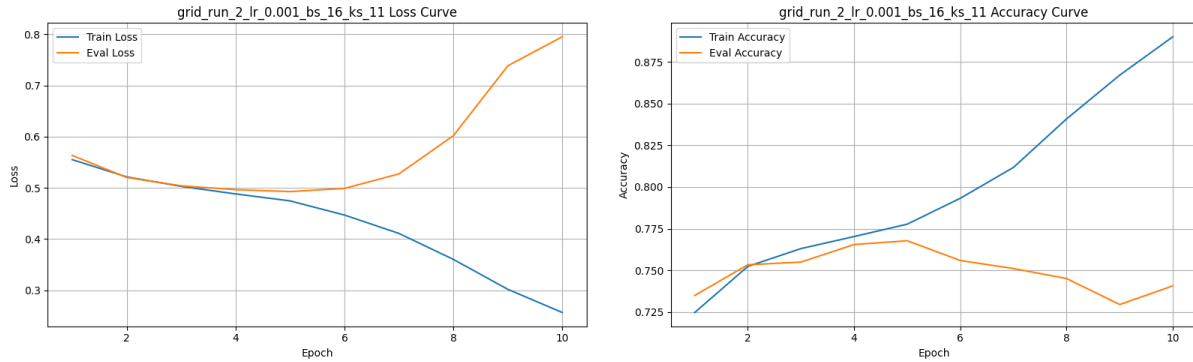


Figure 2: Loss and accuracy curves for the best grid search configuration (Run 2).

9 Task 4: Model Checkpointing

A *Save Best Model* feature was implemented in the training loop.

During training, the test accuracy was monitored after each epoch. If the current epoch achieved a better test accuracy than all previous epochs, the model weights were saved using:

- `torch.save(model.state_dict(), best_model_path)`

This functionality was applied during standard experiment tracking and also during grid search. For each grid-search run, the best-performing model was saved separately.

For the final transfer-learning experiment, the best checkpoint from the grid search was:

- `grid_run_2_lr_0.001_bs_16_ks_11/best_model.pth`

10 Task 5: Frozen Backbone Transfer Learning on Heart

The final task of Project 2 was to perform a frozen-backbone transfer learning experiment using a dataset from a different organ.

10.1 Procedure

The following steps were implemented:

1. Load the Heart dataset using an updated custom loader
2. Load the best pretrained Brain model checkpoint from the best grid-search run
3. Freeze the convolutional backbone:
 - `conv1`
 - `conv2`
 - `conv3`
4. Reinitialize the classifier:
 - `fc1`
 - `fc2`
5. Keep only the classifier trainable
6. Fine-tune the model on Heart for 5 epochs
7. Report the result in TensorBoard

More specifically, only the reinitialized classifier layers (`fc1` and `fc2`) were updated during fine-tuning, while the convolutional backbone remained frozen.

This final version fully followed the frozen-backbone requirement because the early feature extractor was preserved while the final classification layers were reset and retrained for the new organ dataset.

10.2 Frozen Backbone Heart Results

The final transfer-learning experiment with reinitialized classifier produced the following test results on Heart:

- Test Loss = 0.5765
- Accuracy = 0.6935
- Precision = 0.6901
- Recall = 0.7070
- F1-score = 0.6984

10.3 Frozen Backbone Heart Curves

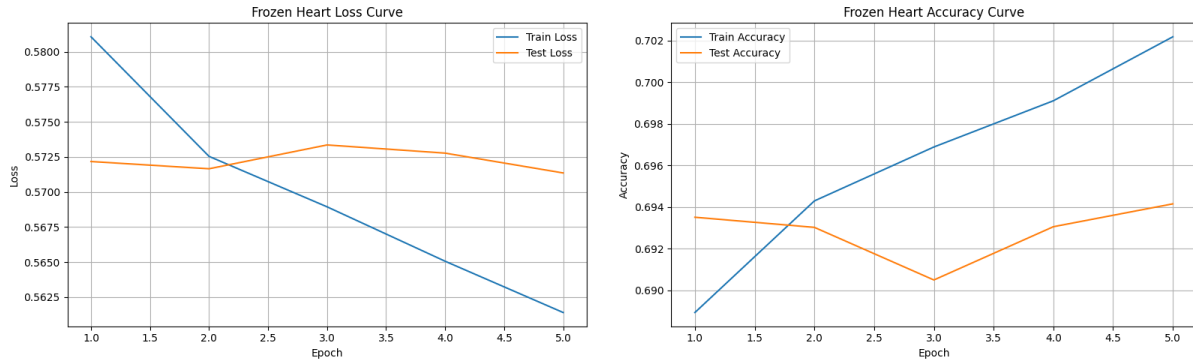


Figure 3: Loss and accuracy curves for frozen-backbone transfer learning on Heart.

10.4 Frozen Backbone Heart Confusion Matrix

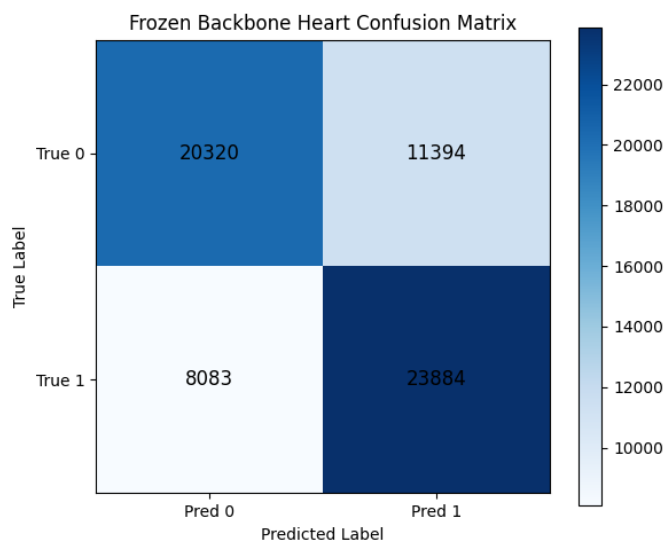


Figure 4: Confusion matrix for frozen-backbone transfer learning on Heart with reinitialized classifier.

11 Discussion

Project 2 transformed the original Project 1 code into a more complete machine learning experimentation workflow.

The major outcomes were:

- TensorBoard successfully tracked model dynamics and provided a visual debugging interface.
- Automated grid search replaced manual hyperparameter tuning.
- Confusion matrices and precision/recall made the evaluation more informative.
- Model checkpointing enabled the best-performing weights to be reused.
- Frozen-backbone transfer learning demonstrated that early features learned on Brain data retained partial usefulness on Heart data.

The grid search showed that the best overall within-dataset setting was:

- Learning Rate = 0.001
- Batch Size = 16
- Kernel Size = 11

This best run achieved a test accuracy of 0.7677, which was stronger than the manually selected within-Brain result in Project 1.

The Heart transfer experiment achieved an accuracy of 0.6935. This lower result is reasonable because Heart is a different organ and therefore introduces domain shift. Even so, the model retained meaningful transfer capability when only the classifier was retrained.

12 Conclusion

This project successfully extended the original DNA accessibility prediction system into a structured experiment-tracking pipeline.

The final implementation completed all required tasks:

- TensorBoard integration
- Automated hyperparameter grid search
- Advanced evaluation with confusion matrix, precision, recall, and F1-score
- Save-best-model checkpointing
- Frozen-backbone transfer learning on a different organ

The most important conclusions are:

- Experiment tracking made the training process more transparent and reproducible.
- Automated hyperparameter search identified a better-performing configuration than manual tuning alone.
- Accuracy alone was not sufficient; confusion matrix, precision, recall, and F1-score were important for proper interpretation.
- Saved pretrained weights could be successfully reused for a new transfer-learning task.
- The frozen-backbone experiment showed partial but meaningful transfer from Brain to Heart.

13 Code Organization

The final Project 2 implementation was organized into the following key files:

- `dataset.py`
- `model.py`
- `train.py`
- `grid_search.py`
- `frozen_backbone_heart.py`

The generated outputs included:

- TensorBoard run folders
- Grid-search result folders
- Best model checkpoints
- Confusion matrix images
- Loss and accuracy curves
- `metrics.txt` summaries

This organization improved readability, reproducibility, and alignment with the project requirements.